

ProjectManagerApp — Hướng Dẫn 03: Services

Package: `com.projectmanager.factory` · `com.projectmanager.service`

TaskFactory.java — Factory Pattern

```
package com.projectmanager.factory;

import com.projectmanager.models.*;

//
// | FACTORY PATTERN
// | Mục tiêu: ẩn logic "tạo object" – caller chỉ biết type
// | Không có Factory: Controller phải if/else new Bug/Feature
// | Có Factory: Controller gọi TaskFactory.create("B")
// | Mở rộng: thêm loại Task mới chỉ sửa Factory, không sửa UI
//
public class TaskFactory {

    private TaskFactory() {} // không khởi tạo – toàn bộ static

    // Tạo task rỗng theo typeCode – Controller tự populate các field
    // typeCode: "B" = Bug, "F" = Feature (không phân biệt hoa/thường)
    public static Task create(String typeCode) {
        return switch (typeCode.toUpperCase()) {
            case "B" -> new Bug();
            case "F" -> new Feature();
            default -> throw new IllegalArgumentException(
                "Loại task không hợp lệ: \"" + typeCode + "\". Chỉ chấp nhận B
hoac F.");
        };
    }
}
```

ProjectService.java — Generic + HashMap + Singleton + IProjectAnalytics

```
package com.projectmanager.service;

import com.projectmanager.models.*;
import com.projectmanager.repository.TaskRepository;
import java.util.*;
import java.util.function.Consumer;
import javafx.application.Platform;

//
// | SINGLETON PATTERN (lần 2 trong app)
//
```

```
// | Lý do: DashboardCtrl, TaskListCtrl, AddTaskCtrl đều cần
// | cùng 1 instance – nếu mỗi controller new riêng thì data
// | không đồng bộ và phải load DB nhiều lần
// |
//
// Generic <T extends Task>: hoạt động với Task, Bug, hoặc Feature
// Singleton instance dùng Task làm upper bound: ProjectService<Task>
// → Add(bug) và Add(feature) đều hợp lệ
//
// Implements IProjectAnalytics: cung cấp
// countByPriority/countByStatus/showSummary
public class ProjectService<T extends Task> implements IProjectAnalytics {

    // — Singleton —————
    private static ProjectService<Task> instance;

    public static ProjectService<Task> getInstance() {
        if (instance == null) instance = new ProjectService<>();
        return instance;
    }

    // reset() gọi khi logout – tránh data cũ của user trước còn tồn tại
    public static void reset() { instance = null; }

    // — Cấu trúc dữ liệu kép —————
    //
    // HashMap<String, T> → findById() trong O(1), kiểm tra trùng ID O(1)
    // ArrayList<T> → giữ thứ tự nhập, duyệt tuần tự cho TableView
    //
    // synchronized(lock): bảo vệ khi loadFromDBAsync() chạy thread riêng
    // đọc/ghi map+list cùng lúc với UI thread → race condition nếu không lock
    private final Map<String, T> map = new HashMap<>();
    private final List<T> list = new ArrayList<>();
    private final Object lock = new Object();
    private final TaskRepository repo = new TaskRepository();

    // — Load từ DB —————

    @SuppressWarnings("unchecked")
    public void loadFromDB() {
        List<Task> dbList = repo.findAll();

        // Build vào temp collections trước – swap vào chính thức trong lock ngăn
        Map<String, T> tempMap = new HashMap<>();
        List<T> tempList = new ArrayList<>();
        for (Task t : dbList) {
            T value = (T) t;
            if (!tempMap.containsKey(value.id)) {
                tempMap.put(value.id, value);
                tempList.add(value);
            }
        }
        synchronized (lock) {
            map.clear(); list.clear();
```

```

        map.putAll(tempMap); list.addAll(templList);
    }
}

// Async: chạy loadFromDB() trong thread riêng → không block JavaFX UI thread
// onComplete: Runnable chạy trên UI thread sau khi load xong (dùng
Platform.runLater)
public Thread loadFromDBAsync(Runnable onComplete) {
    Thread worker = new Thread(() -> {
        loadFromDB();
        if (onComplete != null) Platform.runLater(onComplete);
    }, "load-db-thread");
    worker.setDaemon(true); // JVM không đợi thread daemon khi shutdown
    worker.start();
    return worker;
}

public Thread loadFromDBAsync() { return loadFromDBAsync(null); }

// — Add — finally đảm bảo log luôn chạy (bài 20 – exception handling) —

public boolean Add(T task) {
    try {
        if (task == null) throw new NullPointerException("Task không được
null");
        if (map.containsKey(task.id))
            throw new IllegalArgumentException("ID \"" + task.id + "\" đã tồn
tai");
        if (!repo.insert(task))
            throw new RuntimeException("Lưu xuống DB thất bại");
        synchronized (lock) {
            map.put(task.id, task);
            list.add(task);
        }
        return true;
    } catch (Exception e) {
        System.out.println("[ERROR] " + e.getMessage());
        return false;
    } finally {
        // finally luôn chạy – dù try thành công hay catch được kích hoạt
        System.out.println("[LOG] Thêm task kết thúc.");
    }
}

// — Delete — admin only (kiểm tra ở Controller) —————

public boolean delete(String id) {
    synchronized (lock) {
        T task = map.get(id);
        if (task == null) return false;
        if (!repo.delete(id)) return false;
        list.remove(task);
        map.remove(id);
    }
}

```

```

        return true;
    }

    // — Update Status – cả user và admin đều dùng —————

    // Cập nhật trong DB trước, sau đó cập nhật in-memory để không cần reload
    public boolean updateStatus(String id, String newStatus) {
        T task = map.get(id);
        if (task == null) return false;
        if (!repo.updateStatus(id, newStatus)) return false;
        task.status = newStatus; // cập nhật in-memory – tránh reload toàn bộ
        return true;
    }

    // — Query / Algorithm —————

    public T findById(String id) {
        synchronized (lock) { return map.get(id); } // O(1)
    }

    public List<T> getAll() {
        synchronized (lock) { return new ArrayList<>(list); } // snapshot
    }

    // FilterByStatus – null-safe (bài 20: NullPointerException handling)
    public List<T> FilterByStatus(String status) {
        if (status == null || status.isBlank()) return getAll();
        List<T> result = new ArrayList<>();
        for (T t : getAll()) {
            if (status.equalsIgnoreCase(t.status)) result.add(t);
        }
        return result;
    }

    // Lọc theo loại: "B" = chỉ Bug, "F" = chỉ Feature, null = tất cả
    public List<T> filterByType(String typeCode) {
        if (typeCode == null || typeCode.isBlank()) return getAll();
        List<T> result = new ArrayList<>();
        for (T t : getAll()) {
            if (t.getTypeCode().equalsIgnoreCase(typeCode)) result.add(t);
        }
        return result;
    }

    // Tổng effort async – không block UI khi tính tổng list lớn
    public Thread totalEffortAsync(Consumer<Integer> onResult) {
        Thread worker = new Thread(() -> {
            int total = 0;
            for (T t : getAll()) total += t.GetEffort();
            final int result = total;
            Platform.runLater(() -> onResult.accept(result));
        }, "effort-thread");
        worker.setDaemon(true);
        worker.start();
    }

```

```

        return worker;
    }

    // Two Pointer – đảo ngược danh sách (bài 20 yêu cầu)
    public void reverseOrder() {
        synchronized (lock) {
            int i = 0, j = list.size() - 1;
            while (i < j) {
                T temp = list.get(i);
                list.set(i, list.get(j));
                list.set(j, temp);
                i++; j--;
            }
        }
    }

    // LeetCode #217 – dùng HashSet kiểm tra ID trùng
    public List<String> findDuplicateIds() {
        Set<String> seen = new HashSet<>();
        List<String> dups = new ArrayList<>();
        for (T t : getAll()) {
            if (!seen.add(t.id.toLowerCase()) && !dups.contains(t.id))
                dups.add(t.id);
        }
        return dups;
    }

    // — IProjectAnalytics —————

    // CountByPriority – HashMap đếm tần suất (bài 20 yêu cầu)
    // getOrDefault(key, 0) + 1: lần đầu gặp key trả 0→1, lần sau cộng thêm 1
    @Override
    public Map<String, Integer> countByPriority() {
        Map<String, Integer> result = new LinkedHashMap<>(); // giữ thứ tự chèn
        for (T t : getAll()) {
            result.put(t.priority, result.getOrDefault(t.priority, 0) + 1);
        }
        return result;
    }

    @Override
    public Map<String, Integer> countByStatus() {
        Map<String, Integer> result = new LinkedHashMap<>();
        for (T t : getAll()) {
            result.put(t.status, result.getOrDefault(t.status, 0) + 1);
        }
        return result;
    }

    @Override
    public void showSummary() {
        System.out.println("=".repeat(50));
        System.out.println("  TONG KET DU AN - " + list.size() + " task");
        System.out.println("[ Priority ]");
    }

```

```

        countByPriority().forEach((k, v) -> System.out.printf(" %-8s: %d%n", k,
v));
        System.out.println("[ Status ]");
        countByStatus().forEach((k, v) -> System.out.printf(" %-12s: %d%n", k,
v));
        System.out.println("=".repeat(50));
    }

    // Cung cấp cho DashboardController – tổng số bug, feature, effort
    public int countBugs()      { return (int) getAll().stream().filter(t -> t
instanceof Bug).count(); }
    public int countFeatures() { return (int) getAll().stream().filter(t -> t
instanceof Feature).count(); }
    public int totalEffort()    { int s = 0; for (T t : getAll()) s +=
t.GetEffort(); return s; }
    public int getSize()       { return list.size(); }
}

```

AuthService.java

```

package com.projectmanager.service;

import com.projectmanager.models.dto.LoginRequest;
import com.projectmanager.models.entity.User;
import com.projectmanager.repository.UserRepository;
import com.projectmanager.utils.PasswordHasher;
import com.projectmanager.utils.Validator;

// AuthService: xử lý đăng nhập và đăng ký
// Không chứa SQL – delegate xuống UserRepository
public class AuthService {

    private final UserRepository userRepo = new UserRepository();

    // Login: trả về User nếu thành công, null nếu sai username/password
    public User login(LoginRequest req) {
        Validator.requireNonBlank(req.username, "Username");
        Validator.requireNonBlank(req.password, "Mat khau");

        User u = userRepo.findByUsername(req.username);
        if (u == null) return null; // không
        tìm thấy
        if (!PasswordHasher.verify(req.password, u.passwordHash)) return null; //
        sai password
        return u;
    }

    // Register: trả về true nếu tạo thành công, ném exception nếu lỗi validate
    public boolean register(String username, String password,
        String confirmPassword, String email, String role) {

```

```

Validator.requireNotBlank(username, "Username");
Validator.requireMinLength(username, "Username", 3);
Validator.requireNotBlank(password, "Mat khau");
Validator.requireMinLength(password, "Mat khau", 6);

if (!password.equals(confirmPassword))
    throw new IllegalArgumentException("Mat khau xac nhan khong khop.");

if (userRepo.existsByUsername(username))
    throw new IllegalArgumentException("Username \"" + username + "\" da
ton tai.");

User u      = new User();
u.username  = username;
u.passwordHash = PasswordHasher.hash(password); // hash trước khi lưu
u.email     = (email == null) ? "" : email.trim();
u.role      = (role == null || role.isBlank()) ? "user" : role;
u.status    = true; // tài khoản mới mặc định active

return userRepo.insert(u);
}
}

```

Giải Thích Chi Tiết

1. Tại Sao ProjectService Dùng Cả Singleton VÀ Generic?

Singleton: DashboardCtrl, TaskListCtrl, AddTaskCtrl dùng cùng 1 instance
 → Add task ở AddTaskCtrl → ngay lập tức TaskListCtrl thấy (cùng map + list)
 → Không phải reload DB mỗi khi switch màn hình

Generic <T extends Task>: cùng code hoạt động với mọi Task subclass
 → Hiện tại: ProjectService<Task> – quản lý chung Bug + Feature
 → Mở rộng: ProjectService<Bug> – service chỉ cho Bug (type-safe hoàn toàn)

getInstance() trả về ProjectService<Task>:
 → service.Add(bug) ← Bug extends Task → OK
 → service.Add(feature) ← Feature extends Task → OK

2. Tại Sao Cần **synchronized(lock)** Trong JavaFX?

JavaFX Application Thread

```

TaskListCtrl.initialize()
→ service.getAll()
→ đọc list

```

Background Thread (load-db-thread)

```

loadFromDBAsync()
→ loadFromDB()
→ list.clear() ← RACE CONDITION!
→ list.addAll()

```

synchronized(lock) đảm bảo chỉ 1 thread đọc/ghi list cùng lúc

Snapshot pattern: getAll() lấy bản sao rồi thả lock ngay

→ sort/filter chạy ngoài lock → không giữ lock lâu → ít blocking

3. Background Thread + Platform.runLater

```
// Sai (block UI thread – giao diện đứng khi load DB):
public void initialize(...) {
    service.loadFromDB();           // DB query trên UI thread → app bị đơ
    updateTable(service.getAll()); // chạy sau khi load xong
}

// Đúng (async – UI vẫn mượt):
public void initialize(...) {
    service.loadFromDBAsync(() -> {           // ← callback
        // Platform.runLater: đảm bảo chạy trên JavaFX UI thread
        updateTable(service.getAll());        // cập nhật TableView
        lblStatus.setText("Loaded.");         // cập nhật Label
    });
}
```

4. AuthService — Luồng Login

```
LoginController
→ new LoginRequest(username, password)
→ authService.login(req)
    → Validator.requireNonBlank(...) ← throws IllegalArgumentException nếu
rỗng
    → userRepo.findByUsername(username) ← trả về null nếu không tìm thấy
    → PasswordHasher.verify(plain, hash) ← so sánh SHA-256
← trả về User (thành công) hoặc null (sai thông tin)
→ UserSession.set(user)
→ SceneSwitcher.switchScene("dashboard.fxml")
```